

Cet exercice porte sur les structures de données *FILE* et *PILE*, les graphes et les algorithmes de parcours.

Une agence de voyages organise différentes excursions dans une région de France et propose la visite de certaines villes. Ces excursions peuvent être visualisées sur le graphe non pondéré ci-dessous : les sommets désignent les villes, les arêtes représentent les routes pouvant être empruntées pour relier deux villes. L'agence souhaite proposer un itinéraire permettant de visiter toutes les villes

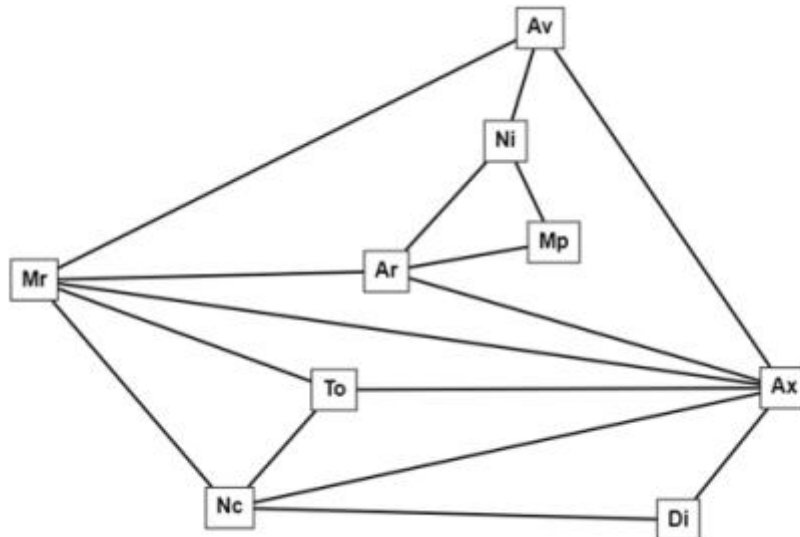


Figure 1. Graphe non pondéré

On choisit d'implémenter un graphe par listes d'adjacence, à l'aide d'un dictionnaire, en langage Python, dont :

- les clés sont les sommets du graphe (type `str`)
- la valeur associée à une clé est la liste des voisins de ce sommet clé.

1. Donner l'implémentation, en langage Python, du graphe de la **Figure 1**. Le dictionnaire obtenu sera stocké dans une variable nommée `G`. Afin de faciliter la notation manuscrite ainsi que la lisibilité, écrire chaque couple clé/valeur sur une nouvelle ligne.

2. Indiquer la signification des lettres dans les acronymes **LIFO** et **FIFO**.

3. Indiquer l'acronyme utilisé pour désigner la structure de **file**.

Voici, en langage Python, les opérations pouvant être effectuées sur une telle file :

- `creerFile()` : renvoie une file vide
- `estVide(F)` : renvoie `True` si la file `F` est vide et `False` sinon
- `enfiler(F, e)` : ajoute l'élément `e` dans la file
- `defiler(F)` : renvoie l'élément à la tête de la file `F` en le retirant de la file `F`.

On donne la fonction `parcours` ci-dessous. Cette fonction prend en paramètres un dictionnaire `graphe` représentant un graphe sous la forme de listes d'adjacence, et une chaîne de caractères `sommet` représentant un sommet du graphe.

```
01 def parcours(graphe, sommet):
02     f = creerFile()
03     enfiler(f, sommet)
04     visite = [sommet]
05     while not estVide(f):
06         s = defiler(f)
07         for v in graphe[s]:
08             if not (v in visite):
09                 visite.append(v)
10                 enfiler(f, v)
11     return visite
```

4. Donner le résultat renvoyé par l'appel `parcours(G, 'Av')`.

5. Donner le nom du parcours obtenu par la fonction `parcours`.

Dans la suite de l'exercice, la distance entre deux sommets désignera le nombre d'arêtes séparant ces deux sommets. Ainsi définie, la distance entre les sommets **Mp** et **Nc** du graphe de la figure 1 est 3.

6. En modifiant la fonction `parcours`, écrire une fonction `distance` ayant pour paramètres `graphe`, un dictionnaire représentant un graphe sous la forme de listes d'adjacence, et une chaîne de caractères `sommet` représentant un sommet du graphe. Cette fonction renverra un dictionnaire tel que :

- les clés sont les sommets du graphe
- la valeur associée à une clé est la distance entre ce sommet clé et le sommet d'origine `sommet`.

7. Donner le résultat renvoyé par l'appel `distance(G, 'Av')`.

On considère maintenant une pile. Voici, en langage Python, les opérations pouvant être effectuées sur une telle pile :

- `creerPile()` : renvoie une pile vide
- `estVide(P)` : renvoie `True` si la pile `P` est vide et `False` sinon
- `empiler(P, e)` : ajoute l'élément `e` au sommet de la pile `P`
- `depiler(P)` : renvoie le sommet de la pile `P` en le retirant de la pile `P`

On donne, ci-dessous, le pseudo-code d'un algorithme de parcours d'un graphe G à partir d'un sommet s :

```
01 créer une pile p
02 empiler s dans p
03 créer une liste visites vide
04 tant que p n'est pas vide
05     x = depiler p
06     si x n'est pas dans la liste visites
07         ajouter x à la liste visites
08         pour chaque voisin v de x
09             empiler v dans p
10         fin pour
11     fin si
12 fin tant que
13 renvoyer visites
```

8. Traduire, dans le corps d'une fonction Python nommée `parcours2`, l'algorithme en pseudo-code donné précédemment. Cette fonction prendra pour paramètres G , un dictionnaire représentant un graphe sous la forme de listes d'adjacence, et une chaîne de caractères s représentant un sommet du graphe.

9. Donner un résultat possible renvoyé par l'appel `parcours2(G, 'Av')`.

10. Donner le nom du parcours obtenu par la fonction `parcours2`.